

Local AI IDE & Agentic Workspace

Unified Offline AI Pair Programmer, Dynamic Task-Based Model Router, Token Optimization Engine, and COSS Startup Strategy.

TARGET SPEC	INFERENCE GATEWAY	BUSINESS MODEL	TARGET LATENCY
i7-1355U • 16GB RAM • RTX 3050 (4GB)	Ollama • llama.cpp • LM Studio	Commercial Open Source (COSS)	< 40ms Tab / < 2s In-line Diffs

1. Executive Summary & Market Opportunity

Modern developer AI tools are polarized: **Cloud IDEs** (Cursor, Windsurf, Copilot) offer slick agentic diffs but leak intellectual property, violate enterprise air-gapping, and trigger continuous subscription costs. Conversely, **Local Chat GUIs** (LM Studio, Jan, Open WebUI) simplify model downloads but lack code editor primitives, AST codebase indexing, and terminal execution.

Cursor / Windsurf Superb multi-file diffs & indexing, but closed-source, privacy-intrusive, and expensive token drains.	LM Studio / Jan 1-click model discovery and clean UI, but cannot edit code on disk, run tests, or manage terminals.	The Unified IDE The "LM Studio + Cursor" hybrid: Native desktop IDE, 100% offline, auto-task router, AST repo maps, and shadow Git.
CLOUD ONLY	CHAT ONLY	THE SOLUTION

2. Hardware Audit & Local Model Compatibility Matrix

Workstation benchmark (13th Gen Intel Core i7-1355U, 16GB System RAM, NVIDIA RTX 3050 Laptop GPU 4GB VRAM) defines the optimal multi-model distribution strategy:

Execution Tier	Model / Quant	Memory Footprint	Generation Speed	Target Task in IDE
100% VRAM Resident	qwen2.5-coder:1.5b (Q4_K_M)	~1.2 GB VRAM	60–80 tok/s	Sub-40ms instant inline Tab autocompletion as you type.
100% VRAM Resident	llama3.2:3b (Q4_K_M)	~2.0 GB VRAM	45–60 tok/s	Docstrings, Git commit summaries, quick regex/rename refactors.
Hybrid GPU + RAM	qwen2.5-coder:7b (Q4_K_M)	~4.7 GB (3GB GPU + 2GB RAM)	15–25 tok/s	High-accuracy multi-file code generation, diff synthesis, debugging.
Hybrid GPU + RAM	deepseek-r1:8b (Q4_K_M)	~4.9 GB (3GB GPU + 2GB RAM)	12–20 tok/s	Deep reasoning, architecture planning mode, algorithm deduction.
Lightweight Embedder	nomic-embed-text	~274 MB RAM	< 20ms	Local codebase semantic search (Vector RAG) for @codebase.
Vision Auxiliary	moondream:2b	~1.7 GB VRAM	30–45 tok/s	UI mockups, screenshot-to-code, diagram interpretation.

3. Deep Competitive Analysis & The 6 Pillars of Differentiation

Existing tools either compromise on privacy (Cloud IDEs) or developer workflow (Chat GUIs). Our architecture resolves the 6 fundamental pain points:

1. Dynamic Router vs. Single-Model Fallacy

Competitors: Force users to lock into one model (slow completions on 8B, dumb reasoning on 1.5B).

Our Solution: Automatically routes typing events to `Qwen-1.5B` (<40ms), edits to `Qwen-7B`, and planning to `DeepSeek-R1`.

2. Tauri Shell (35MB) vs. Electron Bloat (800MB)

Competitors: VS Code forks & extensions consume 500MB–1GB RAM before opening a file.

Our Solution: Native Rust + Tauri shell consumes only ~35MB RAM, saving nearly 1GB of memory exclusively for local LLM weights.

3. Tree-sitter Skeleton vs. Context Choke

Competitors: Dump raw files designed for 200k cloud context, causing 8k local models to hallucinate.

Our Solution: Compresses 50+ files into an 800-token structural AST skeleton of classes, signatures, and exports.

4. 1-Click Model Hub vs. Manual Config Hell

Competitors: Require installing Ollama separately, tweaking ports, and writing JSON configs.

Our Solution: Built-in LM Studio-style downloader with VRAM compatibility badges (● GPU / ● Hybrid / ● OOM).

5. Shadow Git Time-Travel vs. Broken Code

Competitors: Multi-file edits can break working code with messy manual undo.

Our Solution: Background commits to `.ai-checkpoints/` before every agent action with a 1-click snapshot restore button.

6. 100% Air-Gapped vs. Cloud Telemetry

Competitors: Cursor & Copilot leak telemetry; banned by banks, defense, and healthcare.

Our Solution: Verifiable zero-network open-source client; connects to private on-prem GPU clusters (vLLM).

4. Head-to-Head Feature Comparison Matrix

Feature / Capability	Cursor	LM Studio	Continue.dev	Aider	Our Solution
100% Offline & Private (Air-Gapped)	❌ Cloud Only	✅ Yes	✅ Yes	✅ Yes	✅ Native Core
Full Monaco Code Editor UI	✅ Yes	❌ Chat Only	✅ Yes	❌ CLI Only	✅ Monaco Core
Dynamic Task-Based Model Auto-Router	❌ No	❌ No	❌ No	❌ No	✅ Exclusive Feature
1-Click Model Downloader & VRAM Profiler	❌ No	✅ Yes	❌ Manual JSON	❌ CLI Manual	✅ Built-In Hub
Tree-sitter AST Compressed Repo Map	❌ Full Files	❌ None	Partial	✅ Yes (CLI)	✅ Visual IDE Map
Safe Shadow Git Snapshot Rewind	❌ Manual Undo	❌ None	❌ None	Partial	✅ 1-Click Time Travel
Ultra-Lightweight Desktop Shell (<50MB RAM)	❌ Electron (~600MB)	❌ Electron	❌ Extension Host	✅ Python CLI	✅ Tauri v2 (~35MB)

5. Token & KV Cache Optimization Engine (The Secret Sauce)

While cloud models can afford 100,000+ token context dumps, local models on consumer GPUs (e.g. RTX 3050 4GB) are constrained by **KV cache VRAM scaling** and **prefill latency**. Our custom Token Optimization Engine achieves a **90% token reduction** while increasing generation accuracy.

1. Frugal Search/Replace Diff Protocol

The Problem: Asking an LLM to rewrite a 500-line file generates 500 output tokens (~30 seconds on 7B models).
The Fix: The agent enforces a strict `<<<< SEARCH / REPLACE >>>>` grammar, generating only the 5 modified lines (**98% output token savings** & 10x faster response).

2. KV Cache Prefix Freezing (Zero-Latency Prefill)

The Problem: Re-evaluating system prompts, tool schemas, and repo maps on every turn causes noticeable Time-to-First-Token (TTFT) lag.
The Fix: System prompts and AST skeletons are structured as immutable prefixes, allowing Ollama/llama.cpp to **reuse cached KV states (0ms TTFT)**.

3. AST Skeleton Slicing (Semantic Pruning)

The Problem: Sending full helper functions clutters local 8k context windows.
The Fix: Tree-sitter strips full method implementations for non-target files, transmitting only type signatures, docstrings, and class interfaces (**85% input token savings**).

4. Dynamic Context Budgeting & VRAM Guardrails

The Problem: A 32k context allocation on a 7B model can consume 2.5GB VRAM just for the KV Cache, crashing a 4GB GPU.
The Fix: The engine dynamically clamps context windows based on available VRAM (e.g., auto-setting `num_ctx: 8192` with Q8_0 KV Cache quantization).

Hybrid BM25 + Vector Re-Ranking Pipeline

Instead of dumping entire folders into the prompt, the vector engine performs a 2-stage retrieval: **Stage 1** finds the top 15 candidate chunks via embedding similarity; **Stage 2** applies a lightweight lexical BM25 cross-filter, trimming context down to strictly the top 3 most relevant code snippets (saving ~4,000 wasted context tokens per query).

```
// Example: Frugal Diff Protocol Generation Format
FILE: src/auth/jwt.ts <<<<<< SEARCH const token = jwt.sign(payload, secret, {
expiresIn: '1h' }); ===== const token = jwt.sign(payload, secret, { expiresIn: '7d', algorithm: 'HS512' }); >>>>>> REPLACE
```

6. Dynamic Task-Based Auto-Model Router

Rather than using one slow monolithic model, the app embeds an **Intelligent Model Router** that dispatches requests to the optimal parameter size based on real-time IDE events and intent heuristics:



VRAM Lifecycle & Hot-Swapping Logic
The router leverages Ollama's `keep_alive` API. The 1.5B autocomplete model remains permanently pinned in VRAM (`keep_alive: -1`), while heavy 7B/8B models are spawned on-demand and evicted after 3 minutes of inactivity (`keep_alive: "3m"`) to prevent system memory lockups.

7. System Architecture & High-Performance Desktop Stack

```
// Recommended High-Performance Desktop Architecture |— Desktop Shell: Tauri v2 (Rust + React) /* ~15MB binary, 35MB RAM footprint vs 300MB Electron */ |— Editor Core: Monaco Editor (VS Code Engine) /* Syntax highlighting, AST gutters, diff viewer */ |— Terminal Layer: Node-PTY / xterm.js /* Embedded pseudo-terminal with automated error capture */ |— Codebase Index: Tree-sitter + SQLite-vec /* Symbol extraction & local vector similarity */ |— Agent Protocol: MCP (Model Context Protocol) /* Native support for open-source tool calling */ |— Inference Gateway: Ollama API & llama.cpp /* OpenAI-compatible streaming SSE endpoints */
```

8. Advanced Feature Suite & Differentiators

Terminal Auto-Fix Loop
Captures runtime errors (e.g. `pytest` , `cargo`), locates line numbers, and presents a 1-click "Fix & Retry" button.

Generative Live Canvas
Live sandboxed preview panel that immediately renders React components, Tailwind HTML, and Mermaid charts.

Local Whisper Dictation
Hold `Ctrl+Space` to speak natural language refactoring prompts via embedded `whisper.cpp` (100% offline, 75MB RAM).

Native MCP Tool Ecosystem
Supports Anthropic's Model Context Protocol. Connect local models to SQLite, Postgres, GitHub, Playwright, or local documentation servers with 1 click.

Hybrid Cloud (BYOK)
100% offline by default. Optional API keys (Claude 3.7 / GPT-4o) serve as an escape hatch for massive 50+ file refactors.

9. Commercial Strategy & Business Model (COSS Playbook)

The proven monetization framework for developer infrastructure is **Commercial Open Source Software (COSS)**—giving away the core client to drive viral developer adoption while monetizing convenience, team sync, and enterprise security.

Tier	Target Audience	Pricing	Features & Value Proposition
Community (FOSS)	Solo Devs, Students, Hobbyists	100% Free	Full desktop IDE, unlimited local Ollama inference, auto-router, AST repo map, Monaco editor, shadow Git, Token Optimizer. Open-source core builds trust & virality.
Pro (SaaS)	Professional Engineers & Freelancers	\$12 – \$18 / mo	Universal Cloud Fallback (single managed subscription for Claude/GPT/Gemini with zero API key config), cross-device context sync, curated MCP hub.
Enterprise (On-Prem)	Banks, Healthcare, Defense, Tech Firms	\$35 – \$50 / seat / mo	Air-gapped deployment , connects to central on-prem GPU cluster (vLLM), centralized coding policy enforcement (<code>.rules</code> sync), audit logging & SOC2 compliance.

10. Go-To-Market (GTM) & Viral Growth Playbook

1. Open Source Trust

Launch on GitHub with a permissive license. Privacy-conscious devs will inspect code, verify zero telemetry, and advocate virally.

2. Community Seeding

Demo videos on `r/LocalLLaMA`, Hacker News ("Show HN"), and Twitter/X showcasing sub-40ms local tab completions on budget GPUs.

3. Enterprise Inbound

Engineers inside regulated firms (who are banned from Cursor) will champion the tool internally to IT security teams.

11. Actionable 3-Phase Execution Roadmap

Phase 1: MVP Core

- Tauri + React + Monaco Editor
- Ollama API connector & streaming
- Token Optimizer diff grammar
- In-editor code insertion

Phase 2: Agentic Engine

- Multi-file diff review UI
- Tree-sitter AST Repo Map
- Auto-Model Task Router
- Shadow Git checkpoint system

Phase 3: Scale & Launch

- 1-Click model download UI
- Terminal auto-fix loop & MCP
- Launch on GitHub & `r/LocalLLaMA`
- Introduce Pro Hybrid Cloud